Cloud-App DLP (Defender for Cloud Apps) — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Cloud-App DLP (Defender for Cloud Apps): Microsoft Purview 4

The problem . . . 4

The solution . . . 4

What it delivers . . . 4

Business outcomes . . . 5

Boundaries (so scope is clear) . . . 5

1. Solution Overview — Cloud-App DLP (Defender for Cloud Apps) 5

What this solution is . . . 5

Why this matters . . . 6

How it works . . . 6

The two control planes (and what each covers) . . . 7

The pilot→enforce journey (the core discipline) . . . 7

What you can control . . . 8

Limitations (be honest with buyers) . . . 8

Naming convention (used in every doc and script) . . . 8

The standard deployment model . . . 9

2. Requirements — Cloud-App DLP (Defender for Cloud Apps) 9

2.1 Defender for Cloud Apps licensing and portal access . . . 9

2.2 MDCA least-privilege roles . . . 10

2.3 App connectors (API) for the SaaS apps you will govern at rest . . . 10

2.4 Conditional Access App Control routing (for session & access policies) . . . 10

2.5 Classification layer — SITs and sensitivity labels for content inspection . . . 11

2.6 A pilot ring, test-mode dwell, and change control (MDCA-specific) . . . 11

2.7 An alert/incident triage owner and destination . . . 12

Global Requirements — Microsoft Purview Solutions 13

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 13

2. Purview roles (least-privilege) and portal access . . . 13

3. Security & Compliance PowerShell + Graph toolchain . . . 14

4. Data sources & scope in scope . . . 14

5. Compliance, records, privacy & legal stakeholders . . . 14

6. A pilot scope and change control . . . 14

3. Implementation Guide — Cloud-App DLP (Defender for Cloud Apps) 15

Phase 0 — Connect and baseline (Assess) . . . 15

Phase 1 — Enable the two control planes (Configure the plumbing) . . . 16

Phase 2 — Confirm the detection layer (SITs & labels) . . . 16

Phase 3 — Build the policies in TEST mode (alert-only / Monitor-only) . . . 17
Phase 4 — Measure and tune (drive false positives down) . . . 18
Phase 5 — Enforce on the pilot ring (Pilot) . . . 18
Phase 6 — Expand and harden (Enforce → Operate) . . . 19
Order-of-operations summary . . . 19
5. Testing & Validation — Cloud-App DLP (Defender for Cloud Apps) 20
Test A — Detection & control (leaks are controlled) . . . 20
Test B — False-positive rate (adoption gate) . . . 21
Test C — Incident / alert handling (the control is operable) . . . 22
Test-mode-first & rollback validation . . . 23
Evidence to capture (for the deliverable record) . . . 23
6. Troubleshooting — Cloud-App DLP (Defender for Cloud Apps) 24
File policy (data at rest, via API connector) . . . 24
Session policy (real time, via Conditional Access App Control) . . . 25
False positives (the adoption killer) . . . 25
Alerts / incident handling . . . 26
Connectivity / tooling . . . 26
7. Best Practices — Cloud-App DLP (Defender for Cloud Apps) 27
Pilot in test mode before you enforce — always . . . 28
Pick the right plane for the risk . . . 28
Reduce false positives (adoption depends on it) . . . 28
Session-control specifics (know the limits) . . . 28
File-policy / connector specifics . . . 29
Classification hygiene . . . 29
Operations & governance . . . 29
Stay inside the boundary . . . 30
8. Deliverables Summary — Cloud-App DLP (Defender for Cloud Apps) 30
Documentation . . . 30
PowerShell tools (5) — Microsoft Graph / Security & Compliance PowerShell / MDCA REST (read-only) . . . 31
Definition of done . . . 32
Appendix A — PowerShell Scripts 34

Executive Summary — Cloud-App DLP (Defender for Cloud Apps): Microsoft Purview

The problem

Sensitive data does not stay inside Microsoft 365. It lives in the third-party SaaS apps the business runs on — file-sharing (Box, Dropbox, Google Drive), CRM (Salesforce), ITSM (ServiceNow), HR, and more. Native Microsoft Purview DLP protects Exchange, SharePoint, OneDrive, Teams, and endpoints — but not those non-Microsoft clouds. That gap is a compliance exposure and a real data-loss risk: a customer list synced to personal Dropbox, a contract shared "anyone with the link" from Box, or a bulk export pulled from Salesforce onto an unmanaged laptop never touches a Microsoft workload, so native DLP never sees it.

The solution

Extend DLP to third-party cloud apps with **Microsoft Defender for Cloud Apps (MDCA)** — Microsoft's Cloud Access Security Broker — using two complementary controls, and nothing outside the Microsoft stack:

- **File policies** inspect files **at rest** in connected SaaS apps (via each app's API connector), match

their content against Purview **sensitive information types** and **sensitivity labels**, and apply governance — alert, remove external sharing, apply a label, or quarantine.

- **Session policies** apply **real-time** controls as users work in cloud apps (via **Conditional Access

App Control**) — monitor, block download/upload/copy/print, or protect on download.

The defining discipline mirrors the rest of the Purview estate: **every control is piloted in a safe test mode first** — Conditional Access in **report-only**, session policies **Monitor only**, file policies **alert-only** — proven on a **pilot ring**, tuned for a **low false-positive rate**, then enforced and expanded. The result: cloud-app leaks are controlled without blocking the business.

What it delivers

- The **two control planes wired** — target apps connected for at-rest file governance, and onboarded to

Conditional Access App Control for real-time session control.

- **File policies and session policies** that reuse your existing Purview SITs/labels, each proven in test

mode, tuned, and enforced through governance actions or real-time block/protect.

- A **measured false-positive rate** kept at/under an agreed threshold — the difference between a control

people trust and one they switch off.

- An **incident/alert triage workflow** so every match is acted on, surfaced in the Microsoft Defender portal

with optional SIEM/Sentinel forwarding.

• **Five PowerShell tools** (posture discovery, two-plane readiness checker, policy config helper, match/

false-positive report, session-scope helper) — read-only or `-WhatIf` by default, no credentials in code.

Business outcomes

• **The SaaS blind spot closed** — sensitive data is governed where it actually accumulates and leaks, not

only inside Microsoft 365.

• **Minimal disruption** — test-mode-first plus tuning means the control catches leaks without a wave of

false positives, and any session control reverts to report-only instantly if needed.

• **Operable and auditable** — matches raise alerts that are triaged to disposition, with evidence retained.

• **Self-guided at a fraction of consulting cost** — the same method a specialist would use, documented and

tooled to run in-house.

Boundaries (so scope is clear)

Cloud-app DLP is licensing-/plan-dependent — **MDCA, content inspection, and Conditional Access App Control require the appropriate licensing** (verify your tenant), file policies require a **connected app**, and session policies require **CA App Control routing** (with app/activity coverage limits). This solution provides the **technical configuration and the evidence**; *which* apps and data categories are in scope, *which* classifiers legally apply, the **governance/session action per category**, and the **acceptable false-positive threshold** are **compliance, privacy, and legal determinations**, captured alongside the delivered evidence.

1. Solution Overview — Cloud-App DLP (Defender for Cloud Apps)

What this solution is

Sensitive data does not only live in Microsoft 365. It lives in the third-party SaaS apps your business runs on — file-sharing (Box, Dropbox, Google Drive), CRM (Salesforce), ITSM (ServiceNow), HR, and dozens more. Native Microsoft Purview DLP covers Exchange, SharePoint, OneDrive, Teams, and endpoints; it does **not**, on its own, reach into those non-Microsoft clouds. That gap is where regulated data quietly accumulates and leaves.

Microsoft Defender for Cloud Apps (MDCA) — Microsoft's Cloud Access Security Broker (CASB) — closes that gap with two complementary controls:

• **File policies** inspect files **at rest** inside connected SaaS apps (via each app's **API connector**),

match their *content* against **sensitive information types (SITs)** and **sensitivity labels**, and apply **governance actions** — send an alert, notify the owner, remove external/public sharing, apply a sensitivity

label, quarantine, or trash.

- **Session policies** apply controls in **real time** as users interact with cloud apps — through

Conditional Access App Control (a reverse proxy Microsoft Entra routes sessions to) — to **monitor**, **block download/upload/copy/paste/print**, or **protect on download** (apply a label/encryption).

This solution is a repeatable, evidence-driven way to extend DLP to third-party cloud apps using only the **Microsoft Defender portal** (security.microsoft.com → **Cloud apps**), **Microsoft Entra Conditional Access**, **Security & Compliance PowerShell**, and **Microsoft Graph** — no third-party CASB tooling. Its defining discipline mirrors the rest of the Purview estate: **every control is piloted in a safe TEST mode first** — Conditional Access in **report-only**, session policies set to **Monitor only**, and file policies to **alert-only** — proven on a **pilot ring**, tuned for low false positives, and only then moved to enforcement (block / protect / quarantine).

> ***Scope boundary.*** *This solution delivers the **technical method and evidence** for cloud-app DLP. *Which* SaaS apps and data are in scope, *which* classifiers legally apply, and *what* governance/session action a given data category warrants are **business, compliance, privacy, and legal determinations** — deferred to those functions. The solution configures and proves the control; it does not make the policy decision.*

Why this matters

- **Third-party SaaS is the DLP blind spot.** A customer list synced to a personal Dropbox, a contract

shared "anyone with the link" from Box, or a bulk export downloaded from Salesforce onto an unmanaged laptop never touches a Microsoft workload — so native Purview DLP never sees it. MDCA is the control that does.

- **Two problems, two mechanisms.** *Data already sitting* in a SaaS app is a **file-policy** (at-rest,

API-connector) problem; *data moving in a live session* (download to an unmanaged device, upload, copy) is a **session-policy** (real-time, reverse-proxy) problem. Using the wrong one leaves a gap. This solution deploys both, deliberately.

- **Enforcing without a test phase breaks the business.** A session policy set straight to *block download*,

or a file policy set straight to *quarantine*, will stop legitimate work and get switched off. The safe path — report-only Conditional Access, Monitor-only sessions, alert-only file governance, on a pilot ring — is what makes the control survive.

- **It reuses your existing classification investment.** MDCA content inspection uses the same Purview

SITs and **sensitivity labels** that drive your Microsoft-365 DLP, so cloud-app DLP is an *extension* of the classification layer you already have, not a parallel one.

How it works

1. **Connect the apps you want to govern at rest.** In **Cloud apps** → **App connectors**, connect each SaaS

app via its **API connector** (OAuth). The connector lets MDCA scan files, see sharing state, and take governance actions inside that app. *File policies require a connected app.*

1. **Onboard apps for real-time session control.** In **Microsoft Entra** → **Conditional Access**, create a

policy with **Session** → **Use Conditional Access App Control** that routes the target users/apps through the MDCA reverse proxy. Onboarded apps appear under **Cloud apps** → **Conditional Access App Control apps**. *Session and access policies require this routing.*

1. **Define detection with SITs and labels.** MDCA inspects content using Purview **sensitive information

types **(built-in / custom / EDM) via the Microsoft Data Classification Service, and can match on** sensitivity labels**. This is the same classification layer as native DLP.

1. **Build file policies (data at rest).** A **file policy** filters connected-app files by content (SIT),

label, sharing state, owner, or app, and applies **governance actions** — alert, notify, remove sharing, apply label, quarantine, trash — **alert-only first**.

1. **Build session policies (real time).** A **session policy** on a Conditional-Access-App-Control app

controls live activities — download, upload, copy, paste, print, send — with actions **Monitor only**, **Block**, or **Protect** (apply label/encryption on download) — **Monitor only first**.

1. **Pilot in test mode, tune, then enforce.** Run Conditional Access in **report-only**, session policies

in **Monitor only**, and file policies in **alert-only** on a **pilot ring**; measure matches and false positives; then move to enforcement (block / protect / quarantine) and expand.

The two control planes (and what each covers)

Plane	Mechanism	Covers	Requires	Example action
File policy	API connector (at rest)	Files already stored in a connected SaaS app; sharing state	An **app connector** for that app	Remove external sharing; apply label; quarantine
Session policy	Conditional Access App Control (real-time reverse proxy)	Live user activity in a cloud-app session	An **Entra CA policy** routing sessions to MDCA	Block download to unmanaged device; protect on download
Access policy	Conditional Access App Control (real-time)	Whether a session is allowed at all	An **Entra CA policy** routing sessions to MDCA	Block access from unmanaged/non-compliant device

The pilot→enforce journey (the core discipline)

> *Connect apps (API) + onboard apps (CA App Control) → define detection with SITs/labels → build **file policies (alert-only)** and **session policies (Monitor only)** with the CA policy in **report-only**, on a **pilot ring** → measure matches and false positives → **tune** (SIT confidence/instance count, label filters, app/folder scope, exclusions) → enforce on the ring (governance actions on; CA policy on; session*

*action Block/Protect) → validate → **expand** to the full scope → operate (triage alerts, review, re-tune).*

What you can control

- Which **SaaS apps** are connected (file policy) and which are onboarded to **CA App Control** (session).
- Which **SITs / sensitivity labels** each policy matches, at what confidence and instance count.
- Which **users/groups/apps** each session/access policy applies to (via the Conditional Access assignment).
- The **governance action** per file policy (alert → notify → remove sharing → apply label → quarantine →

trash) and the **session action** (Monitor → Block → Protect) — set per compliance's determination.

- The **pilot ring** enforced first and how long you dwell in test mode before enforcing.

Limitations (be honest with buyers)

- **MDCA is licensing-dependent.** Defender for Cloud Apps requires the appropriate plan (available with

Microsoft 365 E5 / E5 Security / E5 Compliance or standalone) — **verify your licensing/tenant.** Content inspection with Purview SITs/labels and Conditional Access App Control likewise depend on your plan.

- **File policies require a connected app; session/access policies require Conditional Access App Control

routing. **Neither works "out of the box" — a file policy is inert until the app's** API connector **is connected; a session policy is inert until an** Entra CA policy** routes sessions to the proxy.

- **Session control (reverse proxy) has coverage and app-compatibility limits.** Some apps, native/desktop

clients, and activities are not fully proxy-controllable — Microsoft maintains the list of featured/ supported apps; **verify the target app is supported** for session control.

- **The portal is evolving.** MDCA now lives inside the **Microsoft Defender portal**

(security.microsoft.com → **Cloud apps**); the legacy standalone portal is being retired. Paths are cited from Microsoft's public docs and flagged "verify your tenant."

- **The governance/session action per data category is a compliance/legal decision**, not a technical

default — this solution configures and proves the control; the determination is deferred.

Naming convention (used in every doc and script)

Every artifact this solution creates uses a consistent `<Mechanism> – <Scope> – <Type>` form, reused verbatim across the docs, scripts, the Defender portal, and Entra. The standard set:

Artifact	Name
App connector (API, data at rest)	`Box – App Connector – API` *(example SaaS app)*

Artifact	Name
Conditional Access App Control policy (Entra)	`MDCA – CA App Control – Session Routing`
File policy (data at rest)	`MDCA – SaaS Files PII – File Policy`
Session policy (real-time)	`MDCA – Download Protect PII – Session Policy`
Access policy (optional, real-time)	`MDCA – Unmanaged Device – Access Policy`
Sensitivity label applied by policy	`Confidential – Applied by MDCA` *(reuse an existing Purview label)*
Pilot ring (users/group)	`MDCA Pilot Ring – Stage1`
Alert/incident workflow	`MDCA Alerts – Triage – Operations`

The standard deployment model

> ***Assess*** *current cloud-app exposure (connected apps, Shadow IT, existing policies) → **Design** which apps, SITs/labels, file policies, and session policies you need → **Configure** connectors + CA App Control routing, then build file policies **alert-only** and session policies **Monitor only** with CA in **report-only** → **Pilot** on a ring (measure → tune) → **Validate** against pass/fail gates → **Expand** to enforcement (governance actions on, CA on, Block/Protect) tenant-wide and operate. Every high-impact change is piloted in test mode first, with a documented rollback.*

2. Requirements — Cloud-App DLP (Defender for Cloud Apps)

These are the **solution-specific** prerequisites for extending DLP to third-party cloud apps with Microsoft Defender for Cloud Apps (MDCA) — **file policies** (data at rest via API connectors) and **session/access policies** (real-time via Conditional Access App Control). They build on — and do not repeat — the six shared prerequisites in `global-requirements-purview.md` (tenant + compliance licensing, least-privilege Purview/compliance roles, S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/ privacy/legal stakeholders, and a pilot scope + change control). Verify each item below before starting the Implementation Guide.

For every requirement: **Description** (what it is), **Why required**, **How to verify**, **How to implement if missing**.

2.1 Defender for Cloud Apps licensing and portal access

• **Description:** Microsoft **Defender for Cloud Apps** must be licensed and provisioned for the tenant.

MDCA is available with **Microsoft 365 E5 / E5 Security / E5 Compliance** or as a standalone plan; content inspection using Purview **SITs/sensitivity labels** and **Conditional Access App Control** likewise depend on the plan. This extends GR-#1 with the MDCA-specific plan detail.

• **Why required:** No file policy, session policy, or content inspection exists without MDCA licensed and turned on. Session control additionally depends on Microsoft Entra Conditional Access (see 2.4).

• **How to verify:** In the **Microsoft 365 admin center** confirm the subscription/add-ons; in the

Microsoft Defender portal (security.microsoft.com) confirm the **Cloud apps** node is present and you can open **Cloud apps** → **Policies**. **Flag "verify your licensing/tenant"** — availability evolves.

• **How to implement if missing:** Acquire/assign the required licensing before building policies; MDCA is

provisioned automatically once licensed — confirm the Cloud apps node appears in the Defender portal.

2.2 MDCA least-privilege roles

• **Description:** Access with the least-privilege MDCA/Defender role the task needs — e.g. **Global Reader**

or a Defender **Security Reader** for read-only discovery, a Cloud App Security **App/Instance admin** or **Security Administrator** to create/manage policies and connectors, and separate **alert/incident** triage where possible. Global Administrator only where genuinely required. Extends GR-#2.

• **Why required:** MDCA and the Defender portal enforce role-based access; connecting apps, creating

policies, and taking governance actions require the matching role, and least privilege limits blast radius.

• **How to verify: Microsoft Defender portal** → **Settings** → **Cloud apps** → **(role/permissions)** and the

Entra role assignments — confirm your account holds the role for the action (read vs configure vs triage).

• **How to implement if missing:** Assign the least-privilege role under change control; separate policy

authors from connector admins and from alert triagers where the org allows.

2.3 App connectors (API) for the SaaS apps you will govern at rest

• **Description:** For **file policies**, each target SaaS app must be connected via its **API connector** in

Cloud apps → **App connectors** (OAuth, using an admin of that app). The connector lets MDCA enumerate files, read sharing state, and apply governance actions. Microsoft supports connectors for a defined set of apps (e.g. Box, Dropbox, Google Workspace, Salesforce, ServiceNow, and others) — **verify the target app has a connector.**

• **Why required:** A **file policy is inert until the app is connected** — MDCA cannot see or act on files

in an app it has no API connector for.

• **How to verify: Cloud apps** → **App connectors** lists connected apps and their **status** (connected /

error); confirm the target app is connected and its initial scan has run.

• **How to implement if missing:** Connect each in-scope app via its API connector using an app admin

account; wait for the initial scan to complete before scoping file policies to it. If an app has no connector, session/access controls (2.4) may still apply, but at-rest file governance will not.

2.4 Conditional Access App Control routing (for session & access policies)

• **Description:** For **session** and **access** policies, target users and apps must be routed through the

MDCA reverse proxy by a **Microsoft Entra Conditional Access** policy with **Session** → **Use Conditional Access App Control**. Onboarded apps then appear under **Cloud apps** → **Conditional Access App Control apps**. Some apps auto-onboard (featured apps); others require a one-time onboarding/deployment step per Microsoft docs.

- **Why required:** Session/access policies operate **only** on sessions that Entra routes to the proxy — no

CA App Control policy means no real-time control.

- **How to verify: Microsoft Entra** → **Protection** → **Conditional Access** shows a policy with the session

control **Use Conditional Access App Control**; **Cloud apps** → **Conditional Access App Control apps** lists the onboarded app(s). The delivered readiness script reports the CA policy via Microsoft Graph.

- **How to implement if missing:** Create the Entra CA policy (start in **report-only**), scope it to the

pilot ring and target app(s), and confirm the app onboards. Verify the target app is supported for session control before relying on it.

2.5 Classification layer — SITs and sensitivity labels for content inspection

- **Description:** MDCA file/session content inspection uses Purview **sensitive information types**

(built-in / custom / EDM) via the Data Classification Service, and can match on **sensitivity labels**. A documented view of *what* sensitive data you are protecting mapped to the SIT/label that detects it is required. Extends GR-#4.

- **Why required:** Policies key off SITs/labels; without knowing which data matters and which detector

finds it, policies are guesswork and generate false positives.

- **How to verify: Purview portal** → **Data classification** → **Classifiers** → **Sensitive info types** lists the

SITs; **Information protection** → **Labels** lists sensitivity labels. The delivered discovery script reports the labels and SITs available for MDCA to use (via S&C PowerShell / Graph). Confirm the data categories with the data owners.

- **How to implement if missing:** Build/confirm the SIT and label inventory before authoring policies; plan

custom SITs / EDM for categories the built-in SITs don't cover (that is a separate Purview task).

2.6 A pilot ring, test-mode dwell, and change control (MDCA-specific)

- **Description:** A named **pilot user/group ring** (e.g. `MDCA Pilot Ring – Stage1`), an agreed **dwell

time **in test mode, and the** change-control window **to move from test to enforce. In MDCA the test-mode equivalents are: the** Entra CA policy in report-only**,** session policies set to Monitor only**, and** file policies set to alert-only** (no quarantine/label/sharing change). Extends GR-#6.

- **Why required:** Cloud-app DLP is high-impact — a *block download* session policy or a *quarantine* file

policy enforced with no baseline stops legitimate work. The house model mandates **test/monitor first**, then a ring, then expand.

- **How to verify:** Confirm the pilot ring membership, that the CA policy is **report-only**, that session

policies are **Monitor only** and file policies **alert-only**, and the approval path to enforce.

- **How to implement if missing:** Define the ring and dwell time and agree change control before enabling

any enforcing action (block / protect / quarantine / remove-sharing).

2.7 An alert/incident triage owner and destination

- **Description:** A named owner (or queue) for **MDCA alerts** — surfaced in **Cloud apps** → **Alerts** and, as

incidents, in the **Microsoft Defender portal** (and via the Graph security alerts API) — plus where matches are reviewed and, optionally, forwarded (SIEM / Microsoft Sentinel). Severity and response actions defined.

- **Why required:** A file/session match with no one to triage it is not a control. Alerts and incidents are

where matches surface; someone must own triage, severity, and response.

- **How to verify:** Confirm the triage owner/queue (`MDCA Alerts – Triage – Operations`), the alert-severity

scheme, and any SIEM forwarding. The delivered report script reads Cloud-Apps alerts via Graph.

- **How to implement if missing:** Assign the owner, define severity/response, and (optionally) wire SIEM

forwarding before enforcement generates real alerts.

Requirements summary

#	Requirement	Gate
GR 1–6	Category prerequisites (`global-requirements-purview.md`)	Tenant/licensing, roles, toolchain, scope, stakeholders, pilot + change control
2.1	MDCA licensing + portal access (verify)	Cloud apps node & policies available
2.2	MDCA least-privilege roles	Configure vs read vs triage roles assigned
2.3	App connectors (API) for target apps	File policies functional (at rest)
2.4	Conditional Access App Control routing	Session/access policies functional (real time)
2.5	SITs + sensitivity labels	Content inspection keys off correct detectors

#	Requirement	Gate
2.6	Pilot ring + test-mode dwell + change control	Report-only/Monitor/alert-first enforced
2.7	Alert/incident triage owner + destination	Matches are acted on

> ***Deferred to compliance/privacy/legal:*** *which SaaS apps and data categories are in scope, which classifiers legally apply, the governance/session action per category, and any regulatory applicability. This solution provides the technical configuration and evidence, not the determination.*

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires (E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Cloud-App DLP (Defender for Cloud Apps)

This is the exact, ordered procedure to extend DLP to third-party cloud apps with Microsoft Defender for Cloud Apps (MDCA) — **file policies** (data at rest, via API connectors) and **session/access policies** (real time, via Conditional Access App Control). Every enforcing control is piloted in **test mode first**: the Entra Conditional Access policy in **report-only**, session policies set to **Monitor only**, and file policies to **alert-only** — then tuned, enforced on a ring, then expanded. Portal paths and cmdlets are from Microsoft's public documentation; **licensing-/plan-dependent features and evolving portal paths are flagged "verify your licensing/tenant"** and are never asserted for your tenant. Do the phases in order. Each phase names its artifacts (from the Overview naming table), states a **pass/fail gate**, and gives a **rollback**.

> ***Automation.*** *The five delivered PowerShell tools run through **Microsoft Graph** (Conditional Access, sensitivity labels, security alerts), **Security & Compliance PowerShell** (`Connect-IPPSSession`, SITs/ labels), and — where a token is supplied via an environment variable — the documented **MDCA REST API** (read-only GET). They are **read-only or** `-WhatIf` **by default**; writes require an explicit `-Commit`. No credentials are stored in code.*
>
> ***Determinations deferred.*** **Which* apps and data categories are in scope, *which* classifiers legally apply, and the *governance/session action* per category are **compliance/privacy/legal** decisions. This guide configures and proves the control; capture those decisions before Phase 4.*

Phase 0 — Connect and baseline (Assess)

1. **Connect the toolchain** (GR-#3). On the admin workstation:

```
Install-Module ExchangeOnlineManagement -Scope CurrentUser -Force   # provides Connect-IPPSSession
Connect-IPPSSession -UserPrincipalName admin@contoso.com            # Security & Compliance PowerShell
Connect-MgGraph -Scopes 'Policy.Read.All','InformationProtectionPolicy.Read','SecurityEvents.Read.All'
```

1. **Run the posture assessment** (delivered script #1, read-only):

```
.\DLPCLOUD-Discovery.ps1 -CsvPath .\DLPCLOUD-Posture.csv
```

It reports the sensitivity labels and SITs available for content inspection, any Conditional Access policies that route sessions to MDCA (CA App Control), and — if an MDCA API token is supplied — connected app connectors and existing file/session policies. Read-only.

1. **Confirm licensing and the two enabling planes** (Req 2.1, 2.3, 2.4): confirm the **Cloud apps** node is

present in the Defender portal; confirm which target apps have an **API connector** (file plane) and whether any **CA App Control** routing exists (session plane). The readiness script (#2) checks both.

Pass/fail gate: you have a written baseline — labels/SITs available, connected apps, and CA App Control state — and confirmed MDCA licensing. **Rollback:** none (read-only).

Phase 1 — Enable the two control planes (Configure the plumbing)

Neither plane works until it is wired. Do the plane(s) your scope needs.

1. **Connect the SaaS apps for file policies (data at rest).** In **Microsoft Defender portal → Cloud apps →

App connectors → Connect an app**, connect each in-scope app (e.g.** `Box – App Connector – API`**) via its API connector, authorizing with an admin of that app. Wait for the** initial scan** to complete — `App connectors` shows the status. *A file policy is inert until its app is connected.*

1. **Onboard apps for session/access policies (real time).** In **Microsoft Entra → Protection → Conditional

Access → Create policy**, create `MDCA – CA App Control – Session Routing`:

- **Assignments** → **Users:** scope to `MDCA Pilot Ring – Stage1` (pilot only, for now).
- **Target resources** → **Cloud apps:** the app(s) to control (e.g. a featured SaaS app).
- **Session** → **Use Conditional Access App Control** → *Use custom policy* (so MDCA session policies apply).
- **Enable policy: Report-only** (this is the test-mode setting — the policy evaluates and logs without

enforcing). Confirm the app appears under **Cloud apps** → **Conditional Access App Control apps**. Verify the target app is **supported for session control** before relying on it.

Pass/fail gate: each in-scope app is **connected** (file plane) and/or **onboarded to CA App Control** with the CA policy in **report-only** (session plane); the discovery/readiness scripts confirm both. **Rollback:** disconnect the app connector (stops at-rest visibility, no data change) / set the CA policy to **Off** (stops routing instantly).

Phase 2 — Confirm the detection layer (SITs & labels)

Do **not** invent SITs/labels — reuse the Purview classification layer you already have.

1. **Confirm the SITs and labels** for each in-scope data category. **Purview portal → Data classification →**

Classifiers → Sensitive info types **(built-in + custom + EDM) and** Information protection → Labels**. The discovery script (#1) lists what is available. If a category has no detector, build the custom SIT/EDM in Purview first (out of scope for this solution's steps, but a hard dependency).

1. **Decide the match per policy:** which **SIT** (at what confidence/instance count) and/or which

sensitivity label each file/session policy will match on.

Pass/fail gate: every in-scope data category maps to a named SIT and/or label that MDCA can inspect. **Rollback:** none (no change — read/confirm only).

Phase 3 — Build the policies in TEST mode (alert-only / Monitor-only)

Create policies with **non-enforcing** actions first. **Cloud apps → Policies → Policy management → Create policy.**

1. **File policy —** `MDCA – SaaS Files PII – File Policy` (data at rest).

Create a **File policy → Filters:** app = the connected app; and **content** filter = *the SIT(s)* from Phase 2 and/or *sensitivity label* = the target label; optionally *sharing level* = external/public. **Governance actions: alert-only first** — enable *Create an alert* and *Notify the file owner*; **do not** yet enable *Remove external sharing*, *Apply sensitivity label*, *Put in quarantine*, or *Trash*. Save.

1. **Session policy —** `MDCA – Download Protect PII – Session Policy` (real time).

Create a **Session policy → Session control type:** *Control file download (with inspection)* → **Activity source / app:** the CA-App-Control app → **content inspection:** match the SIT/label from Phase 2 → **Action: Monitor only** (test mode — no block/protect yet). Save. (A second session policy can cover *upload*, *copy/paste*, or *print* — all **Monitor only** first.)

1. **Access policy (optional) —** `MDCA – Unmanaged Device – Access Policy`**.**

If access-level control is in scope, create an **Access policy** filtering on device tag (unmanaged/non-compliant) with **Action: Monitor only** first.

1. **Keep the CA policy in report-only** (from Phase 1) while session/access policies are **Monitor only** —

together these are the MDCA equivalent of DLP "simulation": everything evaluates and logs, nothing is enforced.

1. **Verify with the config helper** (delivered script #3, `-WhatIf` by default):

```
.\DLPCLOUD-FilePolicy.ps1 -PolicyName 'MDCA – SaaS Files PII – File Policy' -Sit 'Credit Card Number'
```

It validates that the referenced SIT/label exist (S&C/Graph read), emits the exact policy specification to enter in the portal, and — with an MDCA API token — can list current file policies read-only. It never creates an enforcing policy.

Pass/fail gate: file policy exists with **alert-only** governance; session/access policies exist as **Monitor only**; CA policy is **report-only**; scopes and SIT/label matches are correct. **Rollback:** delete the policy (test-mode policies took no enforcing action, so removal is clean).

Phase 4 — Measure and tune (drive false positives down)

Let the policies run in test mode for the agreed **dwell time** (Req 2.6) to capture representative activity.

1. **Review what matched. Cloud apps** → **Alerts** (file-policy alerts, session Monitor-only logs) and the

file/activity views show every match with the file/user, the SIT/label, and the app. Export with the reporting script:

```
.\DLPCLOUD-Report.ps1 -Days 14 -CsvPath .\DLPCLOUD-Matches.csv
```

(Reads Cloud-Apps alerts via the Graph security API; with an MDCA API token it also pulls file-policy matches and activities.)

1. **Classify each match** as true positive or **false positive** and identify the false-positive drivers

(e.g. a built-in *Credit Card* SIT firing on internal reference numbers; a label filter that is too broad).

1. **Tune** to reduce false positives without losing true positives:

• Raise the SIT **confidence level** and/or **instance count** in the content filter.

• Narrow the **app / folder / sharing-level** filter so the policy targets the real exposure.

• Match on a **sensitivity label** instead of a broad SIT where the label is more precise.

• Add **exclusions** (specific owner/site/service account) where justified.

1. **Re-measure** after each change. Repeat until the false-positive rate meets the validation gate

(see Testing).

Pass/fail gate: measured false-positive rate is at/under the agreed threshold on the test sample, with true positives still detected. **Rollback:** revert the last tuning change (the policy is still non-enforcing).

Phase 5 — Enforce on the pilot ring (Pilot)

Now turn on enforcement **for the ring only**, one action at a time.

1. **File policy** → **add governance actions.** On `MDCA – SaaS Files PII – File Policy`, enable the intended

enforcing action per compliance's determination — start with **Remove external/public sharing** or **Apply sensitivity label** (reversible); reserve **Put in quarantine** / **Trash** for where required. Keep the file scope to the pilot ring's apps/sites.

1. **Session policy** → **enforce.** Move `MDCA – Download Protect PII – Session Policy` from **Monitor only** to

Block (block download) or **Protect** (apply label/encryption on download). Move the **Entra CA policy** `MDCA – CA App Control – Session Routing` from **report-only** to **On** — for the pilot ring only (all other users still excluded).

1. **Confirm enforcement is live** for the ring only: a ring user's real-time download of matching content

is now blocked/protected, and a matching at-rest file has its sharing removed / label applied. Users outside the ring are unaffected.

1. **Handle incidents.** Wire `MDCA Alerts – Triage – Operations`: assign the alert owner/queue (Req 2.7),

set severities, and (optionally) forward to SIEM/Sentinel. Triage a real pilot alert end-to-end to prove the workflow.

Pass/fail gate: the ring is enforced (real-time block/protect + at-rest governance), matches raise alerts, the triage workflow produces a dispositioned incident, and nothing outside the ring is enforced. **Rollback:** set the CA policy back to **report-only** / session policy back to **Monitor only** (instant, no data change); disable the file-policy governance actions. *Note: a label/quarantine/trash already applied at rest may need manual reversal — see Troubleshooting.*

Phase 6 — Expand and harden (Enforce → Operate)

1. **Expand scope** from the ring to the full intended population: broaden the **CA policy assignment**

(session) and the **file-policy app/site filter** (at rest), keeping documented exclusions.

1. **Escalate the action** per compliance's determination — e.g. file policy from *remove sharing* to

quarantine, or session from *protect* to *block* — one app/category at a time, re-validating after each.

1. **Add coverage** for more connected apps (file) and more CA-App-Control apps (session) as scope grows,

repeating the test-mode → tune → enforce cycle for each.

1. **Operationalize:** schedule the reporting script for a recurring false-positive/alert review, review

session-policy blocks and file-governance actions, keep an eye on connector health and CA App Control coverage, and re-tune as apps and behavior change.

1. **Validate end-to-end** with the delivered testing procedure (next section) and record the pass/fail

evidence.

Pass/fail gate: enforcement live across the intended apps/users with the compliance-mandated action per category; false-positive rate steady/low; alerts triaged on cadence. **Rollback:** any CA/session policy can be returned to **report-only/Monitor** as an immediate, non-destructive safety switch; file-policy governance actions can be disabled per policy without affecting others.

Order-of-operations summary

Phase	Do	Artifact(s)	Gate	Rollback
0 Assess	Connect, baseline, confirm licensing/planes	Posture CSV	Baseline + readiness confirmed	n/a (read-only)
1 Enable planes	Connect app connectors; create CA App Control policy (report-only)	App connector, CA policy	Both planes wired, CA report-only	Disconnect / CA off

Phase	Do	Artifact(s)	Gate	Rollback
2 Detection	Confirm SITs/labels per category	(reuse Purview SIT/label)	Every category → detector	n/a
3 Build (test)	File policy **alert-only**; session/access **Monitor only**	File/Session/Access policies	All non-enforcing, correct scope	Delete policy
4 Tune	Measure + reduce false positives	Matches CSV	FP rate ≤ threshold	Revert tuning change
5 Pilot	Enforce ring: governance on, CA on, Block/Protect	Enforced ring policies	Ring enforced, incident dispositioned	Back to report-only/Monitor
6 Expand	Broaden scope, escalate action, operate	Enforced policies	Full-scope enforce, low FP, alerts triaged	Report-only / disable

5. Testing & Validation — Cloud-App DLP (Defender for Cloud Apps)

Validate the cloud-app DLP implementation against explicit **pass/fail gates**. The three headline gates are: **(A) cloud-app leaks are controlled** (file governance at rest + real-time session control), **(B) the false-positive rate is low**, and **(C) incidents are handled**. Run every test with **non-real, clearly-fake sample data** (e.g. the Microsoft-documented test credit-card number `4111 1111 1111 1111`) — never real customer/employee data — and record the evidence.

> *All tests assume the policies from the Implementation Guide, first proven in **test mode** (CA policy report-only, session policies Monitor only, file policies alert-only) and then enforced to the **pilot ring**. File-policy tests require a **connected app** (Req 2.3); session tests require **Conditional Access App Control routing** (Req 2.4).*

Test A — Detection & control (leaks are controlled)

A(i) — File policy (data at rest, via API connector)

#	Test	Steps	Pass condition
A1	At-rest match detected	In a **connected** app, place a file containing enough fake SIT instances (or the target sensitivity label) to meet the policy filter.	The file policy matches it; a match/alert appears in **Cloud apps** within the connector scan interval.

#	Test	Steps	Pass condition
A2	Remove-sharing governance	On a matching file shared externally/publicly, let the enforcing file policy act (or run its governance action).	External/public sharing is **removed**; the change is logged as a governance action.
A3	Apply-label / quarantine governance	For a category whose determination is *apply label* or *quarantine*, confirm the action.	The **sensitivity label is applied** / the file is **quarantined** per policy; action logged.

A(ii) — Session policy (real time, via CA App Control)

#	Test	Steps	Pass condition
A4	Block download to unmanaged device	As a pilot-ring user on an **unmanaged** device, open the CA-App-Control app and attempt to **download** content matching the SIT/label.	Download is **blocked** per the session policy; user sees the block message; activity logged.
A5	Protect on download	Where the action is *Protect*, download matching content.	The file is **protected on download** (label/encryption applied) rather than delivered in the clear; logged.
A6	Upload / copy / print (if scoped)	Attempt a scoped activity (upload matching content, copy/paste, or print) in the session.	The activity is **monitored or blocked** per the session policy; logged.
A7	Below-threshold / non-matching negative	Perform the same activities with content that does **not** meet the SIT confidence/instance count or lacks the label.	Correctly **not** controlled — confirms the match tuning holds.

Gate A (pass): A1–A6 each produce the configured control (governance action / block / protect) and a logged event; A7 correctly does not trigger.

Test B — False-positive rate (adoption gate)

#	Test	Steps	Pass condition
B1	Test-mode FP baseline	Over the agreed dwell time (CA report-only, session Monitor-only, file alert-only), export matches (`DLPCLOUD-Report.ps1 -Days <n>`) and classify each as TP/FP.	**False-positive rate ≤ the agreed threshold** (define with compliance; a common target is ≤10% of matches, or an absolute FP count the triage team can absorb).
B2	Legitimate-traffic negative	Run representative legitimate business content/activity (no real sensitive data) through the connected/CA-App-Control apps.	Legitimate traffic is **not** matched — no spurious governance/blocks.
B3	Post-tuning re-measure	After each tuning change (confidence↑, instance count↑, narrower app/folder/sharing filter, label match, exclusions), re-measure.	FP rate trends **down** while true positives (Test A) still fire.
B4	Session UX sanity	In the pilot ring, review real-time **block/monitor** events for legitimate work caught.	Blocks/monitors are on genuine sensitive activity, not routine legitimate work — if routine, re-tune or re-scope.

Gate B (pass): measured FP rate at/under threshold on the test sample with true positives intact.

Test C — Incident / alert handling (the control is operable)

#	Test	Steps	Pass condition
C1	Alert raised	Trigger a file-policy match (A1) or a session block (A4).	An **alert** appears in **Cloud apps** → **Alerts** (and as an incident in the Defender portal / Graph security alerts) with the correct policy, severity, and evidence.
C2	Triage workflow	The `MDCA Alerts – Triage – Operations` owner/queue picks up the alert, sets status, and dispositions it.	Alert moves through the defined states to a resolved disposition by the named owner.

#	Test	Steps	Pass condition
C3	Evidence trail	Locate the same event in the Cloud-apps **activity/file** views (and via `DLPCLOUD-Report.ps1`).	Full evidence trail present (user, app, file, SIT/label, action) for investigation.
C4	SIEM forwarding (if configured)	Confirm the alert reaches the SIEM/Sentinel destination (Req 2.7).	Event present downstream with usable fields.

Gate C (pass): matches raise alerts, alerts are triaged to disposition by the owner, and the evidence trail (and SIEM forward, if used) is present.

Test-mode-first & rollback validation

#	Test	Pass condition
S1	Nothing enforced outside the ring	Users/devices **outside** `MDCA Pilot Ring – Stage1` experience no block/protect/governance during pilot (the CA policy assignment and file scope exclude them).
S2	Instant session rollback	Setting the **Entra CA policy back to report-only** (or the session policy to **Monitor only**) immediately removes real-time enforcement with no data change (verify a previously-blocked download now only monitors).
S3	File-governance rollback	Disabling the file policy's enforcing governance action stops further at-rest changes. *(Already-applied label/quarantine/trash may require manual reversal — record this.)*
S4	Scripts are safe-by-default	`DLPCLOUD-FilePolicy.ps1` run **without** `-Commit` makes no change (`-WhatIf`/spec output only); discovery/readiness/report scripts are read-only.

Evidence to capture (for the deliverable record)

- Baseline posture CSV (Phase 0): labels/SITs available, connected apps, CA App Control state.
- Test-mode match export + TP/FP classification and the computed **false-positive rate** before/after

tuning (Gate B).

• Screenshots/log of Test A file-governance and session block/protect actions and the raised alerts (Gate A/C).

• The dispositioned incident from Test C2 and, if used, the SIEM forward (C4).

• Confirmation that outside-ring users were unaffected (S1) and that report-only/Monitor rollback works (S2), plus any manual at-rest reversal performed (S3).

> ***Determination boundary.*** *These tests prove the* ***control works****. Whether a given category's governance/session action and FP threshold are *acceptable* is a compliance/privacy/legal sign-off, captured alongside the evidence above.*

6. Troubleshooting — Cloud-App DLP (Defender for Cloud Apps)

Symptom-first fixes for the failures you actually hit extending DLP to third-party cloud apps. Most problems are one of: a **file policy** not matching (connector/scan), a **session policy** not engaging (Conditional Access App Control routing), too many false positives, or matches that raise no alert.

File policy (data at rest, via API connector)

A file policy matches nothing even though the file clearly contains sensitive data.

• Confirm the app is **connected and healthy** — **Cloud apps** → **App connectors** shows the status. A file

policy is **inert until the app's API connector is connected** and its **initial scan** has completed. Run `DLPCLOUD-ConnectorReadiness.ps1 -TargetApp <app>`.

• **At-rest scanning is periodic, not instant** — new/changed files can take time to be scanned and matched.

Allow for scan latency before concluding it's broken.

• Check the policy **filters**: app, folder/site, sharing level, and the **content** filter (SIT confidence/

instance count or sensitivity label). Too-strict filters suppress matches — loosen and re-test.

• Confirm the SIT/label the policy references actually exists — `DLPCLOUD-FilePolicy.ps1 ... -WhatIf` reports

VALID/MISSING for each.

The governance action (remove sharing / apply label / quarantine) didn't take effect.

• Confirm the governance action is **enabled** on the policy (during pilot it is deliberately **alert-only**

— that's expected; enable the action in Phase 5).

• Confirm the connector's **permissions** allow that action for that app — some actions require broader app

admin consent; re-authorize the connector if needed.

- Governance runs on matched files going forward and on the next scan — allow time.

Session policy (real time, via Conditional Access App Control)

A session policy never engages (no monitor/block on download/upload).

- The session plane works **only** on sessions Entra routes to the proxy. Confirm a **Conditional Access

policy **with** Session → Use Conditional Access App Control **targets the user and app, and that it is** On** (not still report-only) for enforcement. `DLPCLOUD-ConnectorReadiness.ps1` and `DLPCLOUD-SessionScope.ps1` report this.

- Confirm the app is **onboarded** under **Cloud apps** → **Conditional Access App Control apps** and is a

supported app for session control — some apps, **native/desktop clients**, and activities are not proxy-controllable. Test in a **browser** session first.

- The user must sign in **through the proxy** — an already-established session may not be routed. Sign out/in

or start a fresh session.

Session control breaks the app / users report errors in the proxied session.

- Some apps have known session-control compatibility limits. Verify against Microsoft's featured/supported-app

list; if the app isn't fully supported, fall back to **access policy** (allow/block) or **file policy** (at rest) instead of fine-grained session actions.

- Roll back instantly by setting the CA policy to **report-only** (`DLPCLOUD-SessionScope.ps1 -WhatIf` shows

the rollback step) — no data change.

Everything is enforcing tenant-wide during what should be a pilot.

- The CA policy assignment is too broad. `DLPCLOUD-SessionScope.ps1` flags a policy enforcing for **All

users** — narrow the assignment to `MDCA Pilot Ring – Stage1` and set State back to report-only until ready to expand.

False positives (the adoption killer)

Users are blocked/governed on legitimate, non-sensitive content.

- Export matches with `DLPCLOUD-Report.ps1 -CsvPath ...`, classify TP/FP, and identify the driver.
- **Raise SIT confidence** (medium→high) and/or **instance count** so single/low-confidence hits don't fire.
- **Narrow the filter** — target specific apps/folders/sharing levels/owners rather than "all files".

• **Match on a sensitivity label** where it's more precise than a broad SIT.

• Add a targeted **exclusion** (owner/site/service account) where justified — narrowly, with a documented reason. Re-measure after each change — don't stack changes blind.

Alerts / incident handling

Policy matches occur but no alert appears / no one is notified.

• Confirm the policy has **Create an alert** enabled. Check **Cloud apps** → **Alerts** and the **Microsoft Defender portal** incidents. `DLPCLOUD-Report.ps1` reads Cloud-Apps alerts via the Graph security API.

• Confirm the **triage owner/queue** (`MDCA Alerts – Triage – Operations`) exists and, if forwarding to **SIEM/Sentinel**, that the connector is healthy (Req 2.7).

The report script returns no matches.

• **Latency/dwell:** MDCA alerts and at-rest scans take time; widen `-Days` and re-run.

• Confirm your account holds **SecurityEvents.Read.All** (Graph) for the alerts read, and the right MDCA/ Defender read role.

• Freshly created policies simply may not have matched yet.

Connectivity / tooling

`Connect-MgGraph` **/** `Connect-IPPSSession` **or a cmdlet fails.**

• Ensure the **Microsoft.Graph** and **ExchangeOnlineManagement** modules are installed/updated and you're using a **modern-auth / MFA-capable** admin.

• Confirm the required **Graph scopes** (`Policy.Read.All`, `SecurityEvents.Read.All`, `InformationProtectionPolicy.Read`) are consented, and that your account holds the matching MDCA/compliance role — a missing role surfaces as access denied or empty results.

The MDCA REST calls in the scripts are skipped or fail.

• The REST calls are **optional** — they run only when `$env:MDCA_API_TOKEN` **and** `-McasUrl` are set. If skipped, that's expected; the Graph/S&C paths still work.

• If they fail, verify the **portal URL** (your tenant's MDCA URL) and that the **API token** is valid and has the needed scope. The token is read from the environment only — never hardcode it.

A script made an unexpected change.

• The delivered scripts are **read-only or** `-WhatIf`**/spec-only by default**. `DLPCLOUD-FilePolicy.ps1` never

creates an MDCA policy (there is no supported creation cmdlet) — it only emits a spec. The safe recovery for any enforcement is always to set the CA policy back to **report-only** / the session policy to **Monitor only** in the portal (instant, no data change).

A file was quarantined / labeled / trashed and needs reversing.

- **Quarantine** is recoverable from the app's admin quarantine location (or the user quarantine) — restore

from there.

- An **applied sensitivity label** can be changed by a privileged user/admin per your label policy.
- **Trash** may be recoverable from the app's recycle bin within its retention window — act promptly. Prefer

reversible governance (remove-sharing, apply-label, quarantine) over trash for this reason.

Fast triage table

Symptom	First check
File policy matches nothing	Connector connected/healthy + scan done; filters; SIT/label exists; scan latency
Governance action didn't apply	Action enabled (not still alert-only); connector permissions; allow time
Session policy never engages	CA App Control routing On; app onboarded & supported; browser session; fresh sign-in
Session control breaks app	App not fully supported — fall back to access/file policy; roll back to report-only
Enforcing tenant-wide in pilot	Narrow CA assignment to the pilot ring; set report-only (`DLPCLOUD-SessionScope.ps1`)
Too many false positives	Raise confidence/instance count; narrow app/folder/sharing filter; match on label; exclude
No alert on match	"Create an alert" on policy; Cloud apps → Alerts; triage owner; SIEM connector
Report returns nothing	Latency/dwell; `SecurityEvents.Read.All`; role; widen -Days
Cmdlet/connect fails	Modules updated, modern auth, Graph scopes + MDCA role
MDCA REST skipped/failed	Optional — needs env token + -McasUrl; verify portal URL + token scope

7. Best Practices — Cloud-App DLP (Defender for Cloud Apps)

Field-tested practices for running cloud-app DLP with Microsoft Defender for Cloud Apps so the control stays trusted, effective, and low-noise across third-party SaaS. These complement the Implementation and Testing guides.

Pilot in test mode before you enforce — always

• **Never** switch a new session policy straight to *block* or a file policy straight to *quarantine*. Run the **Entra Conditional Access policy in report-only**, **session policies in Monitor only**, and **file policies in alert-only** first, long enough to capture representative activity (Req 2.6), and read the results before enforcing.

• Treat **report-only / Monitor only as your permanent safety switch**: any session control reverts to report-only in one setting, instantly removing real-time enforcement with no data change — this is your rollback.

• Enforce **one action at a time** on a **pilot ring** (scoped via the CA policy assignment and the file filter) before expanding; escalate *remove-sharing → apply-label → quarantine* and *monitor → protect → block* gradually, re-validating each step.

Pick the right plane for the risk

• **Data already sitting in a SaaS app** (over-shared files, legacy exposure) is a **file-policy** problem — it needs an **API connector** and at-rest governance (remove sharing, apply label, quarantine).

• **Data moving in a live session** (download to an unmanaged device, upload, copy/print) is a **session-policy** problem — it needs **Conditional Access App Control** routing and real-time action.

• Most real programs need **both**. Using only one leaves the other exposure uncovered.

Reduce false positives (adoption depends on it)

• Start at **higher SIT confidence** and a **realistic instance count** — a single card number is noise; ten in one file is a leak.

• **Narrow the filter**: target specific apps, folders/sites, sharing levels (external/public), or owners rather than "all files" — precision cuts false positives dramatically.

• **Match on a sensitivity label** where the label is more reliable than a broad SIT — it reuses a decision already made at classification time.

• Every false positive that governs or blocks legitimate work erodes trust in the whole program. Budget for a tuning cycle, not a one-time build.

Session-control specifics (know the limits)

• **Verify the target app is supported for session control** and that the activities you want (download,

upload, copy, print) are proxy-controllable — native/desktop clients and some apps have gaps. Don't assume coverage you haven't confirmed.

• Roll session controls out **Monitor-first** so you see real activity patterns before blocking; then move scoped activities to **Block/Protect**.

• Prefer **Protect on download** (apply label/encryption) over a hard **Block** where the business still needs the file — it controls the data without stopping the work.

File-policy / connector specifics

• **The connector is the gate.** A file policy is inert until the app's **API connector** is connected and its initial scan has run — verify connector health before blaming the policy.

• Prefer **reversible governance** (remove sharing, apply label) before **irreversible** actions (trash); use **quarantine** (which is recoverable from the quarantine location) rather than trash where you need to hold a file.

• Watch **connector API limits and scan latency** — at-rest governance is periodic, not instantaneous; size expectations accordingly.

Classification hygiene

• **Reuse the Purview SIT/label layer** — cloud-app DLP is an *extension* of your existing classification, not a parallel one. Fewer, well-tuned detectors beat a sprawl of overlapping ones.

• **Version and document** which SIT/label each policy matches (confidence, instance count, filter, owner) so tuning changes are traceable.

Operations & governance

• Give MDCA **alerts an owner and an SLA** (Req 2.7). Match volume without triage is not a control. Forward to **SIEM/Microsoft Sentinel** where central SecOps needs visibility; MDCA alerts also surface as **incidents in the Microsoft Defender portal**.

• Review the false-positive rate, session blocks, and file-governance actions on a **recurring cadence**; re-tune as apps, data, and behavior drift.

• Enforce **least-privilege roles** and, where possible, **separate policy authors from connector admins and from alert triagers**.

• Keep a **change log** of every report-only→on / Monitor→Block / alert-only→governance transition, with the compliance sign-off attached.

Stay inside the boundary

• Cloud-app DLP is **one control**, not a guarantee — session control has app/activity coverage limits and

file inspection sees only content it can classify. Combine with sensitivity labels, endpoint DLP, and insider risk where the risk warrants.

• **Verify licensing/tenant** before relying on MDCA, content inspection, or Conditional Access App Control —

plans and portal paths (now under the **Defender portal** → **Cloud apps**) evolve; don't assume a feature is present.

• The **governance/session action and the acceptable false-positive threshold per category are

compliance/legal determinations** — you provide the technical control and the evidence; they own the decision.

8. Deliverables Summary — Cloud-App DLP (Defender for Cloud Apps)

What you receive in this solution, and what "done" looks like.

Documentation

Document	What it gives you
Solution Overview	The cloud-app DLP model — the two control planes (**file policies** at rest via API connectors, **session/access policies** in real time via Conditional Access App Control), how they reuse Purview SITs/labels, and the pilot→enforce journey; naming convention.
Requirements Guide	Solution-specific prerequisites (MDCA licensing, MDCA roles, app connectors, Conditional Access App Control routing, SIT/label classification layer, pilot ring, alert triage owner) each with verify/implement steps, on top of the six category prerequisites.
Implementation Guide	The exact ordered phases: connect/baseline → enable the two planes → confirm detection → build policies **in test mode** (alert-only / Monitor-only, CA report-only) → tune false positives → enforce to a pilot ring → expand; each phase named, gated, and with a rollback.
Testing & Validation	Pass/fail gates for the three headline outcomes — leaks controlled at rest + real time (A), low false-positive rate (B), incidents handled (C) — plus test-mode-first and rollback checks, all with fake test data.

Document	What it gives you
Troubleshooting Guide	The common failure modes (file policy not matching, connector unhealthy, session control not engaging, CA App Control not routing, floods of false positives, no alerts) and how to resolve them.
Best Practices	Test-mode-first discipline, picking the right plane, false-positive tuning, session-control and connector specifics, classification hygiene, and operations/governance.
Executive Summary	One-page business framing for approvers.

PowerShell tools (5) — Microsoft Graph / Security & Compliance PowerShell / MDCA REST (read-only)

All tools are **read-only or `-WhatIf` by default**; writes require an explicit `-Commit`. No credentials are stored in code — the optional MDCA REST API token is read from an environment variable, never hardcoded. They connect via `Connect-MgGraph`, `Connect-IPPSSession`, and (optionally) the documented MDCA REST API.

#	Script	Role	Default safety
1	**DLPCLOUD-Discovery.ps1**	Cloud-app DLP posture: sensitivity labels + SITs available for content inspection, Conditional Access policies routing sessions to MDCA, and (with an API token) connected app connectors + existing file/session policies.	Read-only
2	**DLPCLOUD-ConnectorReadiness.ps1**	Two-plane readiness checker: **file plane** (app connectors connected/healthy, via MDCA REST if a token is present) and **session plane** (Conditional Access App Control routing, via Graph) — reports whether each is usable.	Read-only

#	Script	Role	Default safety
3	**DLPCLOUD-FilePolicy.ps1**	File/session policy config helper: validates the referenced SIT/label exist, emits the exact policy specification to enter in the portal, and (with a token) lists current policies read-only.	`-WhatIf` (writes need `-Commit`; never creates an enforcing policy)
4	**DLPCLOUD-Report.ps1**	Match / false-positive tuning report: exports Cloud-Apps alerts (Graph security API) and, with a token, file-policy matches/activities for TP-vs-FP classification and rate calculation.	Read-only
5	**DLPCLOUD-SessionScope.ps1**	Session-control scope/health helper: reports the Conditional Access App Control policy's scope (users/groups/apps routed) and its state (report-only vs on), flagging enforcement outside the pilot ring.	Read-only (`-WhatIf` guard on any change guidance)

Definition of done

• The two control planes wired: in-scope apps **connected** (file plane) and/or **onboarded to Conditional Access App Control **(session plane), with the CA policy started in** report-only**.

• SITs/sensitivity labels confirmed for every in-scope data category (reused from Purview).

• File policies (**alert-only**) and session/access policies (**Monitor only**) built and proven in test mode.

• False-positive rate measured and tuned to at/under the agreed threshold (Gate B).

• Enforcement live for the pilot ring — at-rest governance on, CA policy on, session Block/Protect — then

expanded per compliance's action determination (Gate A).

- MDCA alerts raised, triaged to disposition, and evidence trailed (Gate C).
- Test-mode rollback (report-only/Monitor) and safe-by-default scripts verified; change log and compliance

sign-offs captured.

> ***Boundary:*** *the solution delivers the technical control and the evidence. The* ***apps and data categories in scope, the classifiers that legally apply, the governance/session action per category, and the acceptable false-positive threshold*** *are compliance/privacy/legal determinations, captured alongside the evidence.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DLPCLOUD-ConnectorReadiness.ps1

Purpose: Confirms the two enabling planes are ready before you build policies. FILE plane: whether the target SaaS apps have a connected, healthy API connector (via the documented MDCA REST API, only if MDCA_API_TOKEN is set). SESSION plane: whether a Microsoft Entra Conditional Access policy routes the target app/users to MDCA (Conditional Access App Control) and whether it is still in report-only (test) mode (via Microsoft Graph). Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DLPCLOUD-ConnectorReadiness.ps1 -TargetApp 'Box' To also check the file plane, set the token + portal URL (never hardcode): $env:MDCA_API_TOKEN = '<token>' .\DLPCLOUD-ConnectorReadiness.ps1 -TargetApp 'Box' -McasUrl 'https://<tenant>.portal.cloudappsecurity.com' Requires Microsoft Graph PowerShell (Connect-MgGraph). MDCA REST is optional (env token). EXPECTED RESULTS A readiness verdict per plane. PASS (file): the target app's connector is present and connected. PASS (session): a CA policy with CA App Control targets the app. It also flags whether the CA policy is still report-only (expected during pilot) vs already enforcing. .NOTES Purview — Cloud-App DLP. Read-only. No credentials in code; MDCA token read from env var only. > [CmdletBinding()] param([string]$TargetApp, [string]$McasUrl)

How to run: powershell -ExecutionPolicy Bypass -File .\DLPCLOUD-ConnectorReadiness.ps1 -TargetApp 'Box' To also check the file plane, set the token + portal URL (never hardcode): $env:MDCA_API_TOKEN = '<token>' .\DLPCLOUD-ConnectorReadiness.ps1 -TargetApp 'Box' -McasUrl 'https://<tenant>.portal.cloudappsecurity.com' Requires Microsoft Graph PowerShell (Connect-MgGraph). MDCA REST is optional (env token). EXPECTED RESULTS A readiness verdict per plane. PASS (file): the target app's connector is present and connected. PASS (session): a CA policy with CA App Control targets the app. It also flags whether the CA policy is still report-only (expected during pilot) vs already enforcing. .NOTES Purview — Cloud-App DLP. Read-only. No credentials in code; MDCA token read from env var only. > [CmdletBinding()] param([string]$TargetApp, [string]$McasUrl)

DLPCLOUD-Discovery.ps1

Purpose: Builds the baseline for extending DLP to third-party cloud apps: the sensitivity labels and sensitive-information-types available for content inspection (Security & Compliance PowerShell / Graph), the Microsoft Entra Conditional Access policies that route sessions to MDCA (Conditional Access App Control, via Graph), and — only if an MDCA REST API token is supplied via the MDCA_API_TOKEN environment variable — the connected app connectors and existing file/session policies (documented read-only GETs). Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DLPCLOUD-Discovery.ps1 -CsvPath .\DLPCLOUD-Posture.csv Optional MDCA REST read: set the token + your tenant portal URL first (never hardcode): $env:MDCA_API_TOKEN = '<token>' .\DLPCLOUD-Discovery.ps1 -McasUrl 'https://<tenant>.portal.cloudappsecurity.com' Requires Microsoft Graph PowerShell (Connect-MgGraph) and Security & Compliance PowerShell (Connect-IPPSSession). EXPECTED RESULTS Console tables of: sensitivity labels + SIT counts; Conditional Access policies with CA App Control session routing; and (if a token is present) connected app connectors + policy counts. PASS signal: the labels/SITs you intend to inspect on exist, and at least one CA policy routes the target app to MDCA for the session plane (or a connector exists for the file plane). .NOTES Purview — Cloud-App DLP (Defender for Cloud Apps). Read-only. No credentials in code; MDCA token read from env

var only. > [CmdletBinding()] param([string]$CsvPath, [string]$UserPrincipalName, Your MDCA/Defender portal URL, e.g. https://<tenant>.portal.cloudappsecurity.com — only used if MDCA_API_TOKEN is set. [string]$McasUrl)

How to run: powershell -ExecutionPolicy Bypass -File .\DLPCLOUD-Discovery.ps1 -CsvPath .\DLPCLOUD-Posture.csv Optional MDCA REST read: set the token + your tenant portal URL first (never hardcode): $env:MDCA_API_TOKEN = '<token>' .\DLPCLOUD-Discovery.ps1 -McasUrl 'https://<tenant>.portal.cloudappsecurity.com' Requires Microsoft Graph PowerShell (Connect-MgGraph) and Security & Compliance PowerShell (Connect-IPPSSession). EXPECTED RESULTS Console tables of: sensitivity labels + SIT counts; Conditional Access policies with CA App Control session routing; and (if a token is present) connected app connectors + policy counts. PASS signal: the labels/SITs you intend to inspect on exist, and at least one CA policy routes the target app to MDCA for the session plane (or a connector exists for the file plane). .NOTES Purview — Cloud-App DLP (Defender for Cloud Apps). Read-only. No credentials in code; MDCA token read from env var only. > [CmdletBinding()] param([string]$CsvPath, [string]$UserPrincipalName, Your MDCA/Defender portal URL, e.g. https://<tenant>.portal.cloudappsecurity.com — only used if MDCA_API_TOKEN is set. [string]$McasUrl)

DLPCLOUD-FilePolicy.ps1

Purpose: Prepares a Defender for Cloud Apps FILE or SESSION policy safely. It (1) validates that the sensitive-information-type and/or sensitivity label the policy will match on actually exist in the tenant (Security & Compliance PowerShell), and (2) emits the exact, ready-to-enter policy specification (filters + governance/session action) to configure in Cloud apps -> Policies. MDCA file/session policies are not created by a supported PowerShell cmdlet, so this helper does not write policy by default: with -WhatIf it validates and prints the spec only. If an MDCA REST API token is supplied (env var), -WhatIf can also list existing policies read-only. It NEVER emits an enforcing spec by default — the initial spec is alert-only (file) / Monitor-only (session). HOW TO RUN Validate + print a file-policy spec (alert-only) for the Credit Card SIT + Confidential label: powershell -File .\DLPCLOUD-FilePolicy.ps1 -PolicyName 'MDCA - SaaS Files PII - File Policy' `-PolicyType File -App 'Box' -Sit 'Credit Card Number' -Label 'Confidential' -WhatIf Session-policy spec (Monitor only) controlling download: powershell -File .\DLPCLOUD-FilePolicy.ps1 -PolicyName 'MDCA - Download Protect PII - Session Policy'` -PolicyType Session -App 'Box' -Sit 'Credit Card Number' -WhatIf Requires Security & Compliance PowerShell (Connect-IPPSSession) for SIT/label validation. EXPECTED RESULTS -WhatIf: prints VALID/MISSING for each referenced SIT/label and the full policy spec to enter in the portal (test-mode action). PASS signal: all referenced detectors report VALID and the emitted action is non-enforcing (alert-only / Monitor only). -Commit is reserved and refuses to emit an enforcing action. .NOTES Purview — Cloud-App DLP. Validation is read-only; no supported write cmdlet exists for MDCA policies, so this helper emits a spec (never enforcing by default). No credentials in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [ValidateSet('File','Session','Access')][string]$PolicyType = 'File', [string]$App = 'Box', [string[]]$Sit = @('Credit Card Number'), [string]$Label, [int]$MinCount = 10, [ValidateSet('Low','Medium','High')][string]$Confidence = 'High', Non-enforcing test-mode actions only unless the operator deliberately overrides in the portal. [ValidateSet('AlertOnly','MonitorOnly')][string]$Action = $(if ($PolicyType -eq 'File') { 'AlertOnly' } else { 'MonitorOnly' }), [string]$UserPrincipalName, Your MDCA/Defender portal URL — only used to list existing policies read-only if MDCA_API_TOKEN is set. [string]$McasUrl, [switch]$Commit)

How to run: Validate + print a file-policy spec (alert-only) for the Credit Card SIT + Confidential label: powershell -File .\DLPCLOUD-FilePolicy.ps1 -PolicyName 'MDCA - SaaS Files PII - File Policy'

`-PolicyType File -App 'Box' -Sit 'Credit Card Number' -Label 'Confidential' -WhatIf Session-policy spec (Monitor only) controlling download: powershell -File .\DLPCLOUD-FilePolicy.ps1 -PolicyName 'MDCA - Download Protect PII - Session Policy'` -PolicyType Session -App 'Box' -Sit 'Credit Card Number' -WhatIf Requires Security & Compliance PowerShell (Connect-IPPSSession) for SIT/label validation. EXPECTED RESULTS -WhatIf: prints VALID/MISSING for each referenced SIT/label and the full policy spec to enter in the portal (test-mode action). PASS signal: all referenced detectors report VALID and the emitted action is non-enforcing (alert-only / Monitor only). -Commit is reserved and refuses to emit an enforcing action. .NOTES Purview — Cloud-App DLP. Validation is read-only; no supported write cmdlet exists for MDCA policies, so this helper emits a spec (never enforcing by default). No credentials in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [ValidateSet('File','Session','Access')][string]$PolicyType = 'File', [string]$App = 'Box', [string[]]$Sit = @('Credit Card Number'), [string]$Label, [int]$MinCount = 10, [ValidateSet('Low','Medium','High')][string]$Confidence = 'High', Non-enforcing test-mode actions only unless the operator deliberately overrides in the portal. [ValidateSet('AlertOnly','MonitorOnly')][string]$Action = $(if ($PolicyType -eq 'File') { 'AlertOnly' } else { 'MonitorOnly' }), [string]$UserPrincipalName, Your MDCA/Defender portal URL — only used to list existing policies read-only if MDCA_API_TOKEN is set. [string]$McasUrl, [switch]$Commit)

DLPCLOUD-Report.ps1

Purpose: Exports Defender for Cloud Apps alerts so you can classify matches as true vs false positive and compute the false-positive rate during the test-mode dwell (Implementation Phase 4). Primary source is the Microsoft Graph security alerts API filtered to the Cloud Apps service source (microsoftCloudAppSecurity). If an MDCA REST API token is supplied via MDCA_API_TOKEN, it also pulls MDCA alerts/activities directly. Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DLPCLOUD-Report.ps1 -Days 14 -CsvPath .\DLPCLOUD-Matches.csv Optional MDCA REST pull (never hardcode the token): $env:MDCA_API_TOKEN = '<token>' .\DLPCLOUD-Report.ps1 -Days 14 -McasUrl 'https://<tenant>.portal.cloudappsecurity.com' Requires Microsoft Graph PowerShell (Connect-MgGraph, SecurityEvents.Read.All). EXPECTED RESULTS A console summary + optional CSV of Cloud-Apps alerts (title, severity, app, created time). Use it to tag TP/FP and compute the rate. PASS signal (Gate B): after tuning, the FP proportion trends at or under your agreed threshold while true positives are still present. .NOTES Purview — Cloud-App DLP. Read-only. No credentials in code; MDCA token read from env var only. > [CmdletBinding()] param([int]$Days = 14, [string]$CsvPath, [string]$McasUrl)

How to run: powershell -ExecutionPolicy Bypass -File .\DLPCLOUD-Report.ps1 -Days 14 -CsvPath .\DLPCLOUD-Matches.csv Optional MDCA REST pull (never hardcode the token): $env:MDCA_API_TOKEN = '<token>' .\DLPCLOUD-Report.ps1 -Days 14 -McasUrl 'https://<tenant>.portal.cloudappsecurity.com' Requires Microsoft Graph PowerShell (Connect-MgGraph, SecurityEvents.Read.All). EXPECTED RESULTS A console summary + optional CSV of Cloud-Apps alerts (title, severity, app, created time). Use it to tag TP/FP and compute the rate. PASS signal (Gate B): after tuning, the FP proportion trends at or under your agreed threshold while true positives are still present. .NOTES Purview — Cloud-App DLP. Read-only. No credentials in code; MDCA token read from env var only. > [CmdletBinding()] param([int]$Days = 14, [string]$CsvPath, [string]$McasUrl)

DLPCLOUD-SessionScope.ps1

Purpose: Reports the scope and state of the Microsoft Entra Conditional Access App Control policies that drive MDCA session/access controls: which users/groups and apps are routed to the reverse proxy, and whether each policy is still in report-only (test) mode or is enforcing. It flags any policy that is ENFORCING beyond the intended pilot ring so you catch accidental tenant-wide enforcement. Read-only by default; any change to policy state is intentionally left to the portal under change control (the -WhatIf switch only prints what a scope change WOULD be, it never writes). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DLPCLOUD-SessionScope.ps1 -PilotGroupName 'MDCA Pilot Ring - Stage1' Requires Microsoft Graph PowerShell (Connect-MgGraph, Policy.Read.All + Group.Read.All). EXPECTED RESULTS A table of CA App Control policies with State, session type, and included/excluded users/groups. PASS signal: during pilot, the enforcing CA policy is scoped only to the pilot ring (or is still report-only); anything enforcing tenant-wide is flagged RED for review. .NOTES Purview — Cloud-App DLP. Read-only. No credentials in code. State changes are deferred to the portal (change control). > [CmdletBinding()] param([string]$PilotGroupName = 'MDCA Pilot Ring - Stage1', [switch]$WhatIf)

How to run: powershell -ExecutionPolicy Bypass -File .\DLPCLOUD-SessionScope.ps1 -PilotGroupName 'MDCA Pilot Ring - Stage1' Requires Microsoft Graph PowerShell (Connect-MgGraph, Policy.Read.All + Group.Read.All). EXPECTED RESULTS A table of CA App Control policies with State, session type, and included/excluded users/groups. PASS signal: during pilot, the enforcing CA policy is scoped only to the pilot ring (or is still report-only); anything enforcing tenant-wide is flagged RED for review. .NOTES Purview — Cloud-App DLP. Read-only. No credentials in code. State changes are deferred to the portal (change control). > [CmdletBinding()] param([string]$PilotGroupName = 'MDCA Pilot Ring - Stage1', [switch]$WhatIf)